

Contents

Human Resource Management System (HRMS) — Software Requirements Specification	1
Revision history	1
Contents	2
Phases at a glance	2
0. What changed in v2.0	4
1. Introduction	5
2. Overall description	5
3. Architecture and technology stack	6
4. Cross-cutting engineering rules	8
5. Functional requirements	9
6. Corrected workflow specifications	24
7. Data model and database constraints	29
8. API specification	32
9. UI / UX requirements	33
10. Non-functional requirements	33
11. Security, privacy and compliance	34
12. Reporting & analytics	35
13. Deployment, DevOps and CI/CD	35
14. Migration plan from v1.0	36
15. Verification: traceability, acceptance criteria, concurrency tests	37
16. Risks and open questions	37
17. Sign-off	37
Appendix A — Defect register (v1.0 → v2.0)	38
Appendix B — Role-permission matrix (target)	40
Appendix C — Glossary	40

Human Resource Management System (HRMS) — Software Requirements Specification

Version	2.0 — DRAFT for review (supersedes 1.0 baseline)
Date	21 September 2026 (IST)
Author	HRMS Engineering
Repository	github.com/ShubhamvijayJawalkar/CWLiveweb (branch main)
Classification	Confidential — internal distribution only (c) 2026 CWLiveweb
Standards	ISO/IEC/IEEE 29148:2018 (supersedes IEEE 830-1998) · OWASP ASVS 5.0 (target Level 2) · ISO/IEC 25010 · WCAG 2.2 AA
Benchmarks	OrangeHRM 5.x · Sentrifugo · IceHRM · Odoo HR

Revision history

Version	Date	Change
0.1	2026-05-13	Initial generation from codebase analysis
1.0	2026-05-13	Baseline
2.0	2026-09-21	Corrected workflows, target architecture and tech stack, concurrency controls, measurable NFRs, security/compliance, migration plan, defect register (Appendix A)

Status note. v1.0 was marked “Approved for Development” but its sign-off block was empty. v2.0 is a draft until the sign-off table in §17 is completed.

Method note. v2.0 was produced by reviewing the v1.0 document, not by re-reading the code. Where v1.0 describes *current* behaviour (e.g. “Pending approval satisfies the Lunch gate”), the corrected requirement is written as the target, and Appendix A lists the item as a defect **to be verified against the code** before the ticket is closed.

Contents

- **Phases at a glance** (migration roadmap + onboarding/offboarding/ATS process phases — quick reference)
- 0. What changed in v2.0
- 1. Introduction
- 2. Overall description
- 3. Architecture and technology stack
- 4. Cross-cutting engineering rules
- 5. Functional requirements
- 6. Corrected workflow specifications
- 7. Data model and database constraints
- 8. API specification
- 9. UI / UX requirements
- 10. Non-functional requirements
- 11. Security, privacy and compliance
- 12. Reporting and analytics
- 13. Deployment, DevOps and CI/CD
- 14. Migration plan from v1.0
- 15. Verification: traceability, acceptance criteria, concurrency tests
- 16. Risks and open questions
- 17. Sign-off
- Appendix A — Defect register (v1.0 → v2.0)
- Appendix B — Role-permission matrix
- Appendix C — Glossary

Phases at a glance

This section collects every phased plan in the document into one place: the **implementation/migration roadmap** (moving from the v1.0 system to v2.0), and the **process phases** inside the product itself (onboarding, offboarding, ATS pipeline). Each links to its full specification later in the document.

A. Migration roadmap — v1.0 → v2.0 (full detail in §14)

Phase	Name	Content	Exit criterion
0	Freeze & inventory	Freeze schema changes on the DuckDB system; export a full data dictionary; write the target Postgres schema and Alembic baseline migration.	Target schema reviewed and approved

Phase	Name	Content	Exit criterion
1	Dual-write disabled, batch migrate	One-time ETL DuckDB → Postgres with row-count/checksum reconciliation per table; clean up data that violates new constraints.	Reconciliation report shows zero unexplained deltas
2	Service layer cutover	Introduce repository/service layering against Postgres behind a feature flag; run the existing test suite against the new stack; keep DuckDB path available for rollback.	Full regression suite green on Postgres path
3	Cross-cutting rules	Apply CC-01-CC-16 module by module, starting with Attendance/Breaks and Leave (highest concurrency risk), then Payroll, then the rest.	Concurrency test suite (\$15) passes per module
4	New capabilities	Ship FR-JOB-01 (daily attendance), corrected onboarding/offboarding flows, payroll maker-checker, Finance role.	UAT sign-off per module
5	Cutover	Final delta sync, maintenance window, DNS/traffic switch; DuckDB kept read-only as an audit fallback.	Production traffic fully on Postgres path
6	Decommission	Remove /api/v1/* alias after the announced sunset date; archive the legacy DuckDB file per retention schedule.	Legacy path fully retired

Each phase has its own rollback point; phases 1-3 can run in staging with production-shaped data for as long as needed before phase 5 is scheduled.

B. Onboarding — 5 process steps (full detail in §6.5, FR-ONB)

Step	Name	Owner	Advances when
1	Offer Acceptance & Pre-boarding	New hire (via scoped token link, no login required)	≥1 required document uploaded
2	Document Collection & Verification	HR	All required documents Approved
3	System & Access Provisioning	IT	Provisioning task completed → login enabled
4	Workstation & ID Card	Admin/IT	Desk and ID card tasks completed
5	Orientation & Buddy Assignment	Buddy/Manager (confirms — not the new hire)	Orientation confirmed → workflow complete, status Active

C. Offboarding — 5 process stages (full detail in §6.6, FR-OFF)

Stage	Name	Runs	Gates
1	Resignation & Acceptance	Sequential (first)	HR/manager acknowledgement
2	Knowledge Transfer & Manager Clearance	Parallel with Stage 3	Manager sign-off
3	Asset Return & IT Clearance	Parallel with Stage 2	Zero outstanding assets
4	Full & Final Settlement	Sequential (after 2 & 3)	Finance approval (maker-checker)
5	Exit Interview & Access Revocation	Time-triggered	Automatic at 00:00 IST on last working day

D. Recruitment (ATS) pipeline stages (full detail in §6.7, FR-ATS)

Applied → Screened → Interviewed → Offered → Hired, with Rejected/Withdrawn reachable from any stage before Hired. Hired is reachable **only** through an accepted offer (§6.7), which then automatically triggers Onboarding Step 1 above.

E. Document verification workflow

Every document upload (employee self-service, onboarding checklist, expense receipts) passes through the same three-phase check before it is stored: **(1)** content-sniffed MIME type validated against the allowed list for its category, **(2)** size checked against the category cap, **(3)** malware scan — see FR-DOC-02.

0. What changed in v2.0

Flow corrections (the ten that matter most)

1. **Lunch gate** now requires an *Approved*, single-use approval. v1.0 allowed *Pending* to pass, which defeated the approval.
2. **Leave days** use one calculator (working days per employee’s weekly-off pattern, minus holidays) at apply, approve and report time. v1.0 counted calendar days at apply and a different figure at approval. Balances gain a *reserved (pending)* column so approval and cancellation are exact.
3. **Regularization approval now changes attendance data.** In v1.0 approval only updated a status flag.
4. **Daily attendance finalisation job** (Present / Half-day / Absent / Leave / Holiday / Weekly-off) becomes the single source for reports and payroll loss-of-pay. v1.0 had no absence computation and hard-coded Mon–Fri.
5. **Payroll** covers *all* active staff (v1.0 ran only role = Employee), uses one calculation engine (preview and run diverged), integrates LOP/proration and approved reimbursements, and adds maker-checker approval and immutable finalisation.
6. **Onboarding** no longer contradicts itself: v1.0 created Pre-hire users with `allow_login = 0` yet required them to log in and upload documents. Steps now auto-advance on guards; the employee no longer completes their own “team introduction”.
7. **Offboarding** now begins with a real resignation and last-working-day (LWD), runs parallel clearances, is gated on assets returned and settlement approved, and revokes access *on LWD*.
8. **ATS:** Hired can only be reached through an accepted offer; the offer→salary split totals 100% (v1.0’s 50/20/20 left 10% unallocated); accept/reject of a missing offer returns 404.
9. **Expenses:** actor always comes from the session, Paid is restricted to a new **Finance** role, self-approval is blocked, and the state machine is enforced.
10. **Security:** uniform login/reset responses (no account enumeration), account lockout, MFA for privileged roles, enforced CSRF, server-side session revocation, and a **ban on hard-purging audit and payroll records**.

Architecture and stack. PostgreSQL replaces DuckDB as the transactional store (DuckDB stays for analytics). Redis, an object store, a task queue and a dedicated scheduler make the app stateless. See §3.

Document quality. Requirements use “shall”, carry acceptance criteria, and are counted automatically. Contradictions in v1.0 (block/table/test counts, misfiled FRs, role matrix vs end-point table) are resolved. See Appendix A.

1. Introduction

1.1 Purpose

This SRS specifies the HRMS Portal: a web-based, shift-aware, RBAC-driven Human Resource Management System covering employee lifecycle, time and attendance, break governance, leave and regularization, payroll, expenses, helpdesk, assets, documents, on-boarding/offboarding, performance, recruitment (ATS), holidays, notifications, reports and audit.

Readers: product owner, developers, QA, DevOps/SRE, HR and Finance stakeholders, security reviewers, auditors.

1.2 Scope

In scope - Responsive web portal (server-rendered) and a versioned JSON API with OpenAPI. - All modules listed in §1.1, in a single deployable modular monolith. - India-first statutory behaviour (PF, ESI, Professional Tax, TDS) driven by configuration, not code.

Out of scope for v2.0 (tracked in §16): native mobile apps, biometric hardware, SAML/OIDC SSO (designed-for, not delivered), external payroll/bank gateways, multi-country payroll.

1.3 Conventions

- **shall** = mandatory; **should** = recommended; **may** = optional.
- Priority (MoSCoW): **M** Must, **S** Should, **C** Could.
- Δ column: **N** new in v2.0, **C** changed from v1.0, **R** retained.
- Requirement IDs are FR-**<MODULE>-<NNN>**, immutable once published; retired IDs are never reused.
- Counts (tables, endpoints, tests) are generated by CI from the repository and are not hand-maintained in this document.

1.4 Definitions

See Appendix C. Key terms: **shift date** (logical work day keyed to shift start, so night shifts belong to one day), **LWD** (last working day), **LOP** (loss of pay), **F&F** (full and final settlement), **maker-checker** (the person who prepares an action cannot be the person who approves it).

1.5 References

Codebase (app.py, hrms/*, AGENTS.md), OrangeHRM/Sentrifugo/IceHRM/Odoo HR (workflow benchmarks), ISO/IEC/IEEE 29148:2018, OWASP ASVS 5.0, NIST SP 800-63B (password guidance), India Digital Personal Data Protection Act 2023 (privacy obligations; see §11.4).

2. Overall description

2.1 Product perspective

A modular monolith (one deployable, clear internal module boundaries) that replaces spreadsheets and disjoint HR tools. External interfaces: browser (role-gated UI), SMTP/email provider, object storage for files, and (later) SSO and bank/payroll gateways.

2.2 Roles

v1.0 had six roles. v2.0 adds **Finance**, because payroll approval, reimbursement payment and F&F need an owner who is neither HR nor a general admin.

Role	Purpose
Super Admin	System owner; flag <code>is_super_admin</code> (not tied to EMP001); at least two recommended; last one cannot be removed
Admin	Operations administrator; all modules by default; no self-purge
HR	People operations: users, leave, regularization, onboarding/offboarding, ATS, holidays, reports
Finance (<i>new</i>)	Payroll approval, reimbursement payment, F&F approval, payroll rates
IT	Helpdesk (IT queue), assets, onboarding IT tasks
Team Leader (Manager)	Approves direct reports (and delegated reports); team views
Employee	Self-service

Permissions are role defaults plus per-user overrides (`user_permissions`); an explicit deny beats an allow. v1.0's 21 permission modules are retained and **four are added**: `payroll_approve`, `shift_admin`, `policy_admin`, `pii_reveal`. The full matrix is Appendix B.

2.3 Operating environment

Python 3.13 (3.10 reaches end-of-life in October 2026), PostgreSQL 17 or newer, Redis/Valkey, S3-compatible object storage, Linux containers. Browsers: current Chrome, Firefox, Safari, Edge; responsive down to 360 px.

2.4 Design constraints

- Business timezone is Asia/Kolkata; storage is UTC (see CC-02).
- Uploads are limited by type and size per category (see FR-DOC).
- The v1.0 constraint “single DuckDB file, single writer” is **removed**; it contradicted the capacity target.

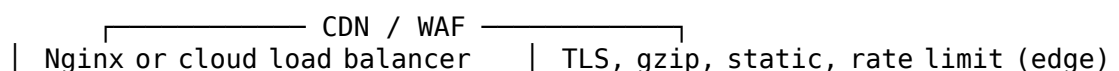
2.5 Assumptions

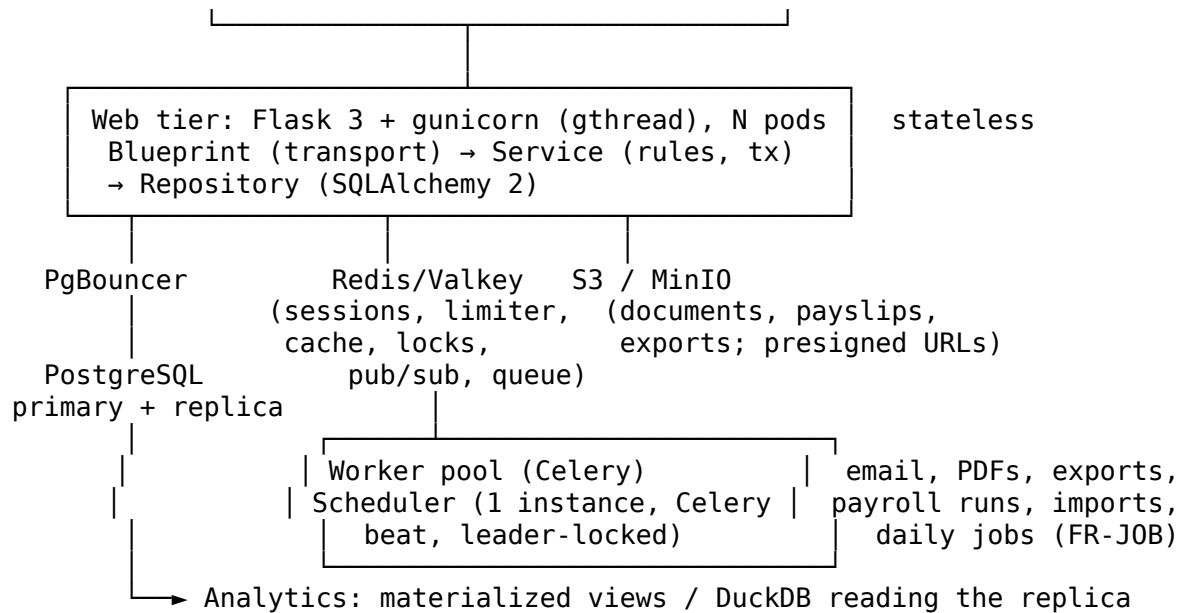
- Shifts are defined per employee with effective dates; night shifts (end < start) roll to the next day.
- Email is optional in dev/test and required in production.
- Statutory values (PF/ESI/PT/TDS) change; they are data with effective dates and must be confirmed with Finance/CA at implementation.

3. Architecture and technology stack

3.1 Architectural decision

Modular monolith, stateless application tier, PostgreSQL as system of record. This keeps the current Flask/Jinja/blueprint code and 250+ tests, fixes the concurrency ceiling, and avoids the operational cost of microservices for a product of this size.





Domain modules (package boundaries, each with its own services and repositories): identity, org, attendance, leave, payroll, expense, recruitment, onboarding, offboarding, performance, servicedesk, documents, assets, notifications, audit, reporting. Modules talk through service interfaces and **domain events**, never by reaching into each other's tables.

3.2 Recommended technology stack

Layer	Choice	Rationale
Language	Python 3.13	Matches existing code and the ruff target; 3.10 is going out of support
Web framework	Flask 3 with gunicorn (gthread)	Keeps the current codebase; threads suit I/O-bound work. Move to uvicorn/ASGI only if async becomes necessary
Validation and OpenAPI	Pydantic v2 models with SpecTree (or APIFlask)	One schema for validation and generated OpenAPI; replaces hand-written flasgger docstrings; enables contract tests
Database	PostgreSQL 17+ (managed service)	Row locks, MVCC, exclusion/partial-unique constraints, NUMERIC, PITR backups. Managed in an India region (e.g. AWS ap-south-1) for latency and data-residency comfort
Access layer	SQLAlchemy 2.0 + psycopg 3	Typed, transaction-scoped sessions; replaces raw SQL strings and the custom <code>_PersistentConnection</code>
Migrations	Alembic	Replaces the custom 15-step <code>migrations.py</code> ; supports expand/contract deploys
Connection pooling	PgBouncer (transaction mode)	Many app workers, few DB connections
Cache, sessions, limiter, locks	Redis / Valkey	Server-side sessions (instant revocation), shared rate limits, cache, distributed locks, SSE pub/sub
Background jobs	Celery + Redis (+ beat, single instance)	Mature; moves email, PDF, Excel, payroll and imports off the request path. Lighter alternative: Procrastinate (Postgres-backed, no extra broker)
Object storage	S3 / MinIO , presigned URLs	Uploads and payslips leave local disk; enables multi-node
Auth and security libs	argon2-cffi , Flask-WTF (CSRF), Flask-Talisman (headers/CSP), pyotp (TOTP), Authlib (OIDC later), Flask-Limiter (Redis backend)	Fixes weak-password, CSRF, header and rate-limit gaps

Layer	Choice	Rationale
Frontend	Jinja2 + HTMX + Alpine.js + Bootstrap 5.3; FullCalendar and Chart.js/ECharts where needed; SSE for live break/notification updates	Keeps server rendering and team skills; replaces polling. A SPA is not justified by current UI needs
PDF / Excel	WeasyPrint (HTML→PDF payslips) or keep reportlab; openpyxl / XlsxWriter	Payslip layout via templates is easier to maintain
File safety	python-magic (MIME sniffing) + ClamAV scan	v1.0 checked extension and size only
Money and time	Python Decimal, NUMERIC(14,2); TIMESTAMPTZ (UTC) + zoneinfo	No floats for money; no naive timestamps
Observability	structlog (JSON logs with request_id), OpenTelemetry, Prometheus + Grafana, Sentry	v1.0 had only optional Sentry and a health endpoint
Testing	pytest, pytest-xdist, testcontainers (per-suite Postgres), Playwright, Locust or k6, Schemathesis, Hypothesis (payroll math)	Removes the “separate DB_FILE per suite” workaround; adds load and contract tests
Quality and supply chain	ruff, mypy/pyright, bandit, pip-audit, gitleaks, Trivy	Only ruff is mentioned in v1.0
Packaging and CI/CD	Multi-stage Docker (non-root), GitHub Actions, Terraform	v1.0 states “CI not present”
Hosting	Start: managed containers (ECS Fargate, Azure Container Apps, or Render) + managed Postgres/Redis. Kubernetes only when scale or org policy requires it	Avoid premature operational load

3.3 Alternatives considered

Option	Verdict
Keep DuckDB for everything	Rejected. Single read-write process, no multi-worker, no separate scheduler container; contradicts the capacity target
SQLite/WAL Full rewrite: FastAPI + React/Next.js	Rejected. Same single-node ceiling, weaker constraint support Only if the team wants an SPA/mobile-first product or has strong TypeScript capacity. Costs months and discards the tested Jinja UI. Choose deliberately, not by default. If chosen, the service and database layers in this SRS carry over unchanged
Microservices	Rejected. Payroll, leave and attendance share transactions and data; splitting adds distributed-transaction risk with no benefit at this scale
NoSQL primary store	Rejected. The domain is relational and needs constraints and transactions

3.4 Where DuckDB fits

DuckDB is retained as an **optional analytics engine**: it ATTACHes the Postgres replica for heavy aggregations (attrition indicators, department summaries, trend reports). Start with Postgres materialized views; add DuckDB only if query time demands it.

4. Cross-cutting engineering rules

These apply to every module. They are the main source of the corrected concurrency and data-integrity behaviour.

ID	Rule
CC-01 Identity	Primary keys shall be BIGINT GENERATED ALWAYS AS IDENTITY (or UUIDv7). Employee codes (EMP###) come from a database sequence. Random gen_id < 2 ³¹ keys are prohibited (collision risk).

ID	Rule
CC-02 Time	Timestamps are stored as TIMESTAMPTZ in UTC. Shift-date logic uses the organisation timezone Asia/Kolkata. API responses use ISO-8601 with offset. (v1.0 stored naive IST and appended +05:30 on read.)
CC-03 Money	Money is NUMERIC(14,2) in the database and Decimal in code; rounding is half-up to the paisa. Floats are prohibited for money.
CC-04 Atomic transitions	Every status change shall be a conditional update, e.g. UPDATE ... SET status='Approved' WHERE id=:id AND status='Pending', and the caller shall check the affected-row count. Zero rows means 409 (already actioned).
CC-05 Invariants in the database	Uniqueness and non-overlap rules shall be enforced by constraints (see §7.3), not only by application checks.
CC-06 Balances	Balance changes shall be conditional updates or SELECT ... FOR UPDATE inside the same transaction as the status change.
CC-07 Idempotency	Create/action POSTs that a UI may double-submit (start-break, leave apply, expense submit, payroll run) shall accept an Idempotency-Key header, stored for 24 h; repeat calls return the original result.
CC-08 Optimistic locking	Editable records carry a version integer. PUT requires the current version; mismatch returns 409 with the current state.
CC-09 Transactions and side effects	One transaction per use case in the service layer. Emails, notifications and audit events are written to a transactional outbox in the same transaction and delivered by workers with retry and dead-letter handling.
CC-10 Actor from session	The acting user is always taken from the authenticated session. Request bodies shall never supply emp_id for “self” actions.
CC-11 Central authorization	All access decisions go through one policy function can(actor, action, resource) with scopes <i>self, direct reports (incl. delegates), department, all</i> . Decorators call it; the matrix in Appendix B is tested against it.
CC-12 Input validation	Schema validation (Pydantic) at the boundary; server is authoritative; unknown fields rejected on writes.
CC-13 Errors	Uniform envelope: {"error":{"code":"LEAVE_OVERLAP","message":"...","details":{"..."},"request_id":"..."} API routes never redirect; 401 = unauthenticated, 403 = authenticated but not permitted.
CC-14 Files	Files are stored under random keys in object storage; original filename is metadata only; access is via authenticated endpoint or short-lived presigned URL.
CC-15 Configuration	12-factor: configuration and secrets come from environment or a secrets manager. No secrets, databases, WAL files, uploads or .secret_key in version control.
CC-16 Retention	Every table containing personal data has a documented retention period and an anonymisation path (see §11.4).

5. Functional requirements

Format: FR-<MOD>-<NNN> — statement — priority — Δ. Full acceptance criteria live in the test suite (§15); this section states the corrected rule.

FR-AUTH — Authentication & session

ID	Requirement	Pri	Δ
FR-AUTH-01	Login by employee code (case-insensitive, trimmed) and password. Rate limit 10/min per account and per IP, Redis-backed.	M	C

ID	Requirement	Pri	Δ
FR-AUTH-02	On failed login, the response is identical whether the account is unknown, blocked, archived or the password is wrong: 401 {"error": "invalid_credentials"}. No 403 variants that reveal account state.	M	N
FR-AUTH-03	10 consecutive failures within 15 minutes locks the account for 15 minutes and notifies the user by email.	M	N
FR-AUTH-04	Sessions are server-side (Redis-backed), 8 h idle timeout, 24 h absolute timeout, HttpOnly, SameSite=Lax, Secure outside local dev. Logout invalidates the server-side session immediately.	M	C
FR-AUTH-05	Logout closes the active user_sessions row: total_hours = (logout – login) / 3600; audited as LOGOUT.	M	R
FR-AUTH-06	GET / redirects to /dashboard if authenticated, else /login.	M	R
FR-AUTH-07	GET /dashboard renders the admin or self dashboard by role; unauthenticated requests get 302 (page) or 401 (API, Accept: application/json).	M	R
FR-AUTH-08	Forgot-password always returns 202 {"message": "If the account exists, a reset link has been sent."} regardless of match, to prevent enumeration. Rate limit 5/min per IP and per email.	H	C
FR-AUTH-09	Reset tokens: token_urlsafe(32), hashed at rest, 1 h expiry, single use, invalidated on any successful reset for that account.	H	C
FR-AUTH-10	Password policy: minimum 10 characters, checked against a breached-password list (e.g. Have I Been Pwned k-anonymity API or an offline corpus); no forced complexity rules or expiry, per NIST SP 800-63B. v1.0's 6-character minimum is a defect (Appendix A-01).	M	C
FR-AUTH-11	MFA (TOTP) is required for Super Admin, Admin and Finance roles, and optional for others.	H	N
FR-AUTH-12	CSRF: a csrf_token cookie plus X-CSRF-Token header (double-submit) is enforced on all mutating requests (POST/PUT/PATCH/DELETE) to /api/*. v1.0 issued the token but never checked it (Appendix A-02).	M	C
FR-AUTH-13	Credential listing (emp_id, name, role, department) is restricted to Super Admin, requires re-authentication within the last 5 minutes, and is fully audited.	L	C

ID	Requirement	Pri	Δ
FR-AUTH-14	A scheduled job purges expired reset tokens hourly and auto-closes breaks Active for more than 12 hours (→ Orphaned, duration_minutes = NULL, ended_reason = 'orphan_timeout'). Reason is recorded, not defaulted to zero, so reports can exclude it.	H	C

FR-USR — User management

ID	Requirement	Pri	Δ
FR-USR-01	List GET /api/users: pagination (page, per_page, max 200), filters (search, role, status, department, active), sort on an allow-listed column set; server-side only.	M	R
FR-USR-02	Create POST /api/users: emp_id matches EMP\d{3,}, name, email (RFC 5322-plausible, case-insensitive unique), role in the role table, department. Default status Active, allow_login = true. Leave balances are seeded from the leave policy for the employee's grade/location , not a hard-coded 12/10/20. Welcome email carries a 24 h single-use reset token.	M	C
FR-USR-03	Update GET/PUT /api/users/<id>: field-level allow-list; role changes are audited with old/new value; email change re-verifies uniqueness.	M	R
FR-USR-04	Block/unblock: POST /api/users/<id>/block unblock; blocking force-revokes all active sessions for that user (Redis).	H	C
FR-USR-05	Archive DELETE /api/users/<id> → Archived; self-archive 409. Restore → Active.	H	R
FR-USR-06	Purge is removed from the product. Replace with anonymisation: POST /api/users/<id>/anonymize (Super Admin, two-person approval, see FR-USR-06a) scrubs direct identifiers (name→"Former Employee #id", email, phone, address, bank details) but retains audit_log, payroll_items and statutory records for the legal retention period (§11.4). Hard-deleting audit and payroll data (v1.0 FR-USR-06) is prohibited (Appendix A-03).	H	C
FR-USR-06a	Anonymisation requires a second Super Admin's approval (maker-checker) before it executes.	H	N

ID	Requirement	Pri	Δ
FR-USR-07	Bulk POST /api/users/bulk {action, emp_ids[]}: self excluded; per-row result reported, partial failure does not fail the batch.	H	R
FR-USR-08	Meta, CSV export (async job, \$CC-09, for exports > 500 rows), sessions history (last 50).	L	R
FR-USR-09	Permissions GET/PUT /api/users/<id>/permissions: full replace is audited with a diff (added/removed modules), not just “updated”.	H	C
FR-USR-10	Bulk import POST /api/users/import: CSV via pandas, processed as a background job with a progress endpoint (GET /api/imports/<job_id>); per-row validation; report {imported, skipped, errors[:50], job_id}.	H	C
FR-USR-11	Dependents: self CRUD, ownership enforced (emp_id = session.emp_id), not the payload.	M	C
FR-USR-12	Employee-submitted documents route through the same validation as FR-DOC-02 (MIME sniff, size, malware scan).	L	C
FR-USR-13	Profile GET/PUT /api/profile: self-edit fields are a strict subset of admin-edit fields (e.g. an employee cannot change their own role, department, manager_emp_id, or salary-linked fields).	M	C
FR-USR-14	Change password requires the current password and re-authenticates the session token.	M	R
FR-USR-15	Navigation is rendered from the same policy function as API authorization (CC-11) — one source of truth, not a separate hand-maintained nav rule set.	M	C

FR-ATT — Attendance, breaks, shift

ID	Requirement	Pri	Δ
FR-ATT-01	Break types are configurable per organisation/location (seed: Tea 15 min, Lunch 60 min, Personal 30 min); Lunch.requires_approval = true by default and is configurable.	M	C

ID	Requirement	Pri	Δ
FR-ATT-02	Start break POST /api/breaks/start {break_type}, idempotency key required : 403 if allow_breaks = false; daily quota checked against Completed breaks since shift_start(employee, today); if requires_approval, an Approved (not Pending) approval for today's shift date must exist, else 403; any existing Active break is auto-ended (Completed, ended_reason='auto_end_new_break') before the insert; the whole operation is one transaction with a partial unique index preventing two Active rows for one employee (CC-05).	M	C
FR-ATT-03	End break POST /api/breaks/<id>/end: ownership enforced by employee id, not just findability; duration_minutes = (end – start)/60; audited.	M	R
FR-ATT-04	List own breaks, scoped to the current shift window.	M	R
FR-ATT-05	Break approval request POST /api/break-approvals {break_type}: Lunch only ; one Pending per employee per shift date enforced by a unique partial index, not just an app check.	H	C
FR-ATT-06	Approve/reject: allowed if actor is the employee's manager (including an active delegate, FR-LEA-08a) or has role HR/Admin; conditional update (CC-04); audited; notifies employee.	H	C
FR-ATT-07	Break-type summary per type: minutes used since shift start, whether approval is required, latest approval status for the shift date.	L	R
FR-ATT-08	Login-hours query for a given shift date.	M	R
FR-ATT-09	Shift summary: shift_hours = last_logout – first_login (not sum of sessions) when both exist, else now – first_login for an open shift, capped at the scheduled shift length +25% to avoid a forgotten-logout skewing the figure — flagged estimated: true in that case. productive_hours = shift_hours – break_minutes/60; efficiency = productive_hours / shift_hours × 100.	H	C
FR-ATT-10	<i>(retired — folded into FR-ATT-09; see Appendix A)</i>	—	—
FR-ATT-11	Monthly calendar: sessions, breaks, leaves (day-expanded), holidays, and now computed daily status (FR-JOB-01) for each day.	H	C
FR-ATT-12	Live monitoring (admin): employees currently on an Active break.	L	R

ID	Requirement	Pri	Δ
FR-ATT-13	Break summary (admin): per-employee active/completed counts and total minutes for the current shift window.	L	R
FR-ATT-14	Recently disposed breaks (admin), last hour.	L	R
FR-ATT-15	Dashboard stats are computed by a cached query (Redis, 15 s TTL) refreshed by a worker, not recomputed per request; JSON keys are stable (total, logged_in_today, on_break, blocked, pending_leaves).	M	C
FR-ATT-16	Admin break console: active/dispensed/summary in one call; admin dispose ends any Active break with an audited reason.	H	R
FR-ATT-17	Shift configuration (shift_start, shift_end, Fixed/24x7, weekly-off pattern) is effective-dated (shift_assignments table), not a bare column on users, so a shift change does not rewrite history.	M	C

FR-REG — Regularization

ID	Requirement	Pri	Δ
FR-REG-01	List with pending_my_approval, status, month filters; manager view includes delegated reports (FR-LEA-08a).	M	R
FR-REG-02	Create {date, reason, requested_login_time?, requested_logout_time?}: future dates rejected; a specific corrected time is required , not just a text reason, so approval has something concrete to apply (Appendix A-04); one Pending per employee per date, enforced by a unique partial index.	M	C
FR-REG-03	Approve shall write the corrected time(s) into the day's session/attendance record and trigger FR-JOB-01 to recompute that day's status. v1.0 approval only flipped a status flag and never touched attendance data (Appendix A-05) . Reject leaves attendance untouched. Cancel: Pending only, by owner or admin.	H	C
FR-REG-04	Export to Excel (async for large ranges); page shows admin/manager context.	M	R

FR-LEA — Leaves

ID	Requirement	Pri	Δ
FR-LEA-01	List with pending_my_approval / admin filters / self filters, as v1.0, plus delegated-manager visibility.	M	C
FR-LEA-02	Create {leave_type, start_date, end_date, session (Full/First-half/Second-half), reason}; dates parsed and swapped if reversed; days requested = working days in range per the employee's weekly-off pattern, minus organisation holidays in range (the same function used at approval and reporting — see FR-LEA-09); balance check is remaining >= requested inside the same transaction as the insert (CC-06); overlap check against Pending/Approved leaves is a database exclusion constraint (CC-05), not just an app query; on success, leave_balance.reserved += requested and the row is inserted Pending.	M	C
FR-LEA-03	Export to Excel (async for large ranges).	M	R
FR-LEA-04	Approve: actor is manager (or delegate) or HR/Admin, not the applicant; conditional update Pending→Approved; leave_balance.used_days += days, leave_balance.reserved -= days, in one transaction.	H	C
FR-LEA-05	Reject: conditional update; leave_balance.reserved -= days. Cancel: Pending only, or Approved with a future start date (with the same reserved/used reversal), by owner or admin.	H	C
FR-LEA-06	Balance query returns {total, used, reserved, remaining} per type per year; remaining = total – used – reserved.	M	C
FR-LEA-07	Grants: HR/Admin can add days to one or more employees' balances for a type/month/year; fully audited as LEAVE_GRANT with before/after totals.	H	R
FR-LEA-08	Leave policy (accrual rate, carry-forward cap, encashment rule, weekly-off pattern) is configuration per grade/location, versioned by effective date — not a hard-coded 12/10/20 in the seed data.	M	N
FR-LEA-08a	Approval delegation. A manager can delegate approval authority to another employee for a date range (e.g. while on leave). Delegates appear in pending_my_approval views and their approvals are audited as "approved by delegate for manager X".	S	N

ID	Requirement	Pri	Δ
FR-LEA-09	The working-day / holiday-deduction function used to compute leave days (FR-LEA-02), the payroll LOP calculation (FR-PAY-04), and the reports “working days” figure (FR-RPT) is the same function, called from one place. v1.0 used three different day-counting rules across these three areas (Appendix A-06).	M	N

FR-PAY — Payroll

ID	Requirement	Pri	Δ
FR-PAY-01	Pages/APIs for salary structures, payroll rates, payroll runs — Finance and Admin roles; HR has read access.	M	C
FR-PAY-02	Salary structure: {emp_id, basic, hra, allowances[], deductions[], effective_from}; multiple structures per employee are allowed with non-overlapping effective ranges (exclusion constraint). Preview endpoint uses the exact same calculation function as the payroll run (FR-PAY-05) — one function, not two (Appendix A-07).	M	C
FR-PAY-03	Payroll rates (PF/ESI/PT/TDS) are versioned, effective-dated configuration, editable by Finance, fully audited; a run always uses the rates effective on the run's period end date.	M	C
FR-PAY-04	Calculation: gross = basic + hra + Σallowances; pf = min(gross × pf_rate, pf_max); esi = gross × esi_rate if gross ≤ esi_max else 0; pt = pt_amount if gross > pt_threshold else 0; TDS per the configured slab table (not hard-coded percentages); LOP is deducted using FR-LEA-09's working-day function against the days marked Absent/Half-day by FR-JOB-01; approved, unpaid reimbursements for the period are added to net pay. net = gross – (Σdeductions + pf + esi + pt + tds) – lop_amount + reimbursements. All amounts NUMERIC(14,2).	M	C

ID	Requirement	Pri	Δ
FR-PAY-05	Run POST /api/payroll - runs {month, year}: duplicate (month, year, status != 'Cancelled') prevented by a unique constraint; executes as a background job covering every employee with status Active or Onboarding whose salary structure is effective for the period (v1.0 covered only role = Employee, silently excluding managers and HR — Appendix A-08); created in status Draft; progress is pollable.	M	C
FR-PAY-06	Maker-checker approval. Draft → Submitted (Finance) → Approved (a different Finance/Admin user) → Finalized. A run cannot be approved by the user who submitted it. Once Finalized, payroll items are immutable; corrections require a new run (an adjustment run) referencing the original. v1.0 had no approval step (Appendix A-09).	H	N
FR-PAY-07	Payslip (JSON and PDF), self or admin/Finance; payslip_generated flag set on first PDF generation; PDF stored in object storage, served via presigned URL.	H	C
FR-PAY-08	Bank file (CSV: employee id, name, net salary, account, IFSC) and TDS report, Finance/Admin only, only for Finalized runs.	H	C
FR-PAY-09	Review-cycle progress query, and a quarterly scheduled job (FR-JOB) opens the next review cycle.	S	R

FR-EXP — Expenses

ID	Requirement	Pri	Δ
FR-EXP-01	Categories are configurable (seed: 6 as v1.0).	M	R
FR-EXP-02	Create {cat_id, amount, description, receipt}: emp_id is always taken from the session (CC-10); v1.0 accepted an emp_id override in the request body, allowing a claim to be filed as someone else (Appendix A-10) ; category existence validated. Inserted Pending.	M	C
FR-EXP-03	Status transitions are a strict state machine: Pending → Approved/Rejected by manager or HR (self-approval blocked: actor ≠ claim owner); Approved → Paid by Finance only (not “any logged-in user” as in v1.0 — Appendix A-11); each transition is a conditional update and audited.	M	C

FR-TKT — Tickets / helpdesk

ID	Requirement	Pri	Δ
FR-TKT-01	Queues (HR, IT), department-scoped subcategories, SLA target per priority (configurable).	M	C
FR-TKT-02	Create/list with the same visibility rule as v1.0 (owner, assignee, matching department, or admin); notifies the department and Super Admin.	M	R
FR-TKT-03	Detail view enforces the same visibility rule server-side, not just in the list query (defence in depth). Comments append with updated_at bump.	M	C
FR-TKT-04	Status transitions Open → In Progress → Resolved → Closed, and Reopened (new) if a closed ticket receives a comment from the reporter within 7 days. Assignment audited.	M	C

FR-DOC — Documents

ID	Requirement	Pri	Δ
FR-DOC-01	List scoped to owner unless HR/Admin.	M	R
FR-DOC-02	Upload: multipart; MIME type is sniffed from content (not trusted from the filename extension), size cap per category, and the file is scanned for malware before being persisted to object storage under a random key. v1.0 validated only extension and size (Appendix A-12).	M	C
FR-DOC-03	Download: owner or HR/Admin, via presigned URL, audited. Delete: owner or Admin only (v1.0 allowed any authenticated user — Appendix A-13).	M	C

FR-ONB — Onboarding

See §6.5 for the corrected end-to-end flow; this table lists the endpoint-level requirements.

ID	Requirement	Pri	Δ
FR-ONB-01	Five steps: Offer Acceptance & Pre-boarding → Document Collection & Verification → System & Access Provisioning → Workstation & ID Card → Orientation & Buddy Assignment. A pre-hire record can view/upload documents via a scoped pre-boarding token link without a normal login, because allow_login = false until step 3. v1.0 required the same pre-hire user to log in to upload documents while login was disabled — a contradiction (Appendix A-14).	M	C
FR-ONB-02	Onboarding page/API for HR/Admin with full visibility; candidate/new-hire view is token-scoped.	M	C
FR-ONB-03	Checklist creation is automatic on offer acceptance (event-driven from FR-ATS-04), not a manual HR API call that can be skipped or duplicated; a unique constraint prevents two active checklists per employee.	M	C
FR-ONB-04	Document upload/review as v1.0 (size gates, Approve/Reject with mandatory notes on reject), through the FR-DOC-02 validation pipeline.	M	C
FR-ONB-05	Step advancement is guard-based, automatic once its guard is satisfied, and HR is only asked to confirm physical/logistics steps (workstation ready, ID card issued) — the system does not ask the new hire to “complete their own introduction” as the terminal action (v1.0’s step 5 — Appendix A-15). Step 5 (orientation) is marked complete by the assigned buddy or manager.	M	C
FR-ONB-06	Status query: own workflow, or (HR/Admin) the list of everyone currently onboarding, with days-in-step.	M	R

FR-OFF — Offboarding

See §6.6 for the corrected flow.

ID	Requirement	Pri	Δ
FR-OFF-01	Triggered by a resignation record {emp_id, notice_date, last_working_day, reason, initiated_by} — resignation is a first-class object, not implied. Five stages: Resignation & Acceptance → Knowledge Transfer & Manager Clearance → Asset Return & IT Clearance → Full & Final Settlement → Exit Interview & Access Revocation. Stages 2 and 3 run in parallel , not sequentially (Appendix A-16), both gating stage 4.	M	C
FR-OFF-02	Tasks per stage, assignable to specific owners (manager, IT, Finance); exit interview scheduling.	M	R
FR-OFF-03	Stage completion is a conditional update per stage with its own guard (assets returned = 0 outstanding; settlement approved by Finance); access revocation (allow_login = false, sessions killed, permissions cleared) executes automatically at 00:00 IST on the last working day , not only when someone manually finishes stage 5 (Appendix A-17). users.status → Inactive.	M	C

FR-PERF — Performance

ID	Requirement	Pri	Δ
FR-PERF-01	Goals CRUD, rating 1-5 by manager (not self), transition to Completed.	M	C
FR-PERF-02	Review cycle create/list restricted to HR/Admin; submit requires the reviewer to be the assigned reviewer for that review (v1.0 allowed any authenticated user to submit any review — Appendix A-18). 360° feedback: reviewer cannot be the subject.	M	C

FR-HOL — Holidays

ID	Requirement	Pri	Δ
FR-HOL-01	National/Optional types, per-location applicability; list with search/filter.	M	R
FR-HOL-02	CRUD (HR/Admin, holidays permission), duplicate (name, date) per location prevented by a unique constraint; copy year-to-year with Feb-29 handling; import/export; iCal feed.	M	R

ID	Requirement	Pri	Δ
FR-HOL-03	Opt-in/opt-out for Optional holidays, one active opt-in per employee per holiday (unique constraint); approval queue for HR.	M	R

FR-ATS — Recruitment

See §6.7 for the corrected flow.

ID	Requirement	Pri	Δ
FR-ATS-01	Stages: Applied → Screened → Interviewed → Offered → Hired, with Rejected/Withdrawn reachable from any stage before Hired. Pipeline view aggregates counts per job per stage.	M	C
FR-ATS-02	Jobs and candidates CRUD (HR/Admin); candidate status change via the same stage machine (no arbitrary jump to Hired).	M	C
FR-ATS-03	Convert-to-employee is not a manual action — it is a consequence of POST /api/offers/<id>/accept (FR-ATS-04). A candidate reaches Hired only through an accepted offer, closing the v1.0 gap where a candidate could be marked Hired directly without ever having an offer (Appendix A-19).	M	C
FR-ATS-04	Offers: {candidate_id, offered_salary, basic_pct, hra_pct, allowances_pct} where the three percentages sum to 100 (validated) — v1.0's fixed 50/20/20 split silently discarded 10% of salary (Appendix A-20). Accept → candidate Hired, creates the salary structure and (via a domain event) triggers onboarding checklist creation (FR-ONB-03). Reject/Accept on a missing or already-decided offer returns 404/409, not a silent no-op (v1.0's idempotent-but-wrong 200 — Appendix A-21).	M	C

FR-JOB — Scheduled and computed jobs (*new module*)

ID	Requirement	Pri	Δ
FR-JOB-01	Daily attendance finalisation , nightly per shift date: for every active employee, classify the day as Present, Half-day, Absent, On Leave, Holiday or Weekly-off, from sessions, approved leave, holidays and the employee's weekly-off pattern (not a hard-coded Mon-Fri — Appendix A-22). Written to attendance_days, the single source for reports and payroll LOP. Idempotent: re-running for a date replaces that date's rows in one transaction.	M	N
FR-JOB-02	Hourly: purge expired reset tokens; close orphaned breaks (FR-AUTH-14).	H	R
FR-JOB-03	Quarterly: open the next performance review cycle (FR-PAY-09).	S	R
FR-JOB-04	Nightly: revoke access for employees whose last working day was today (FR-OFF-03).	M	N
FR-JOB-05	The scheduler runs as exactly one active instance (leader election via a Redis or Postgres advisory lock), independent of how many web or worker pods are running. v1.0's in-process APScheduler duplicated jobs under multiple gunicorn workers (Appendix A-23).	M	N

FR-ANL — Analytics

ID	Requirement	Pri	Δ
FR-ANL-01	Headcount, leave trends, expense summary, performance summary — as v1.0, served from materialized views refreshed on a schedule, with a documented “as of” timestamp on the response.	M	C
FR-ANL-02	Attrition risk score: the heuristic weights (leave frequency, regularization frequency, early-break frequency, attendance ratio) are configuration , reviewed with HR before go-live, not fixed constants (Appendix A-24), and the endpoint states the score is a heuristic indicator, not a prediction, in its response metadata.	S	C

FR-RPT — Reports

ID	Requirement	Pri	Δ
FR-RPT-01	Self-service and admin/HR views; self-service is enforced by ignoring any emp_id/department the caller supplies and substituting the caller's own scope server-side (v1.0 already did this — retained and covered by an explicit test, Appendix A-25 notes it as “verify, not assumed”). “Working days” uses FR-LEA-09's shared function.	M	C
FR-RPT-02	Excel export (async for ranges > 1 month or > 200 employees) and department summary.	M	C

FR-AUD — Audit

ID	Requirement	Pri	Δ
FR-AUD-01	Every mutating action writes an audit row {actor, action, entity, entity_id, before, after, ip, request_id, created_at} via the transactional outbox (CC-09); the audit log is never purged by the application (only by an explicit, documented, legally-driven retention job — see §11.4); audit_log is excluded from FR-USR-06 anonymisation for the retention period, then the actor reference is generalised, not deleted.	M	C

FR-NOT — Notifications

ID	Requirement	Pri	Δ
FR-NOT-01	List (last 50) with unread count; delivered in-app and by email per category preference, via the outbox (CC-09) so a slow SMTP provider cannot block the request that triggered the notification.	M	C
FR-NOT-02	“Mark as read” sets is_read = true and does not delete the row — v1.0's /read endpoint deleted rows, so read notifications vanished instead of being marked (Appendix A-26). A separate, explicit “clear” action may delete.	M	C
FR-NOT-03	Preferences per category (Onboarding, Leaves, Expenses, Tickets, Payroll, Tickets-SLA), {in_app, email} each, default true.	S	R

FR-AST — Assets

ID	Requirement	Pri	Δ
FR-AST-01	Issue/return, own-assets view, admin/IT list and CRUD; return sets Returned/return_date; outstanding asset count feeds the offboarding gate (FR-OFF-03).	M	C

6. Corrected workflow specifications

6.1 Authentication & session lifecycle

Employee → POST /login {emp_id, password}

- lookup by emp_id (case-insensitive)
- verify Argon2 hash
- any failure (unknown id, bad password, blocked, archived, allow_login=false)
 - identical 401 {"error":"invalid_credentials"} [FR-AUTH-02]
 - failed-attempt counter += 1 (Redis, 15 min window)
 - 10 failures → account locked 15 min, email sent [FR-AUTH-03]
- success:
 - create server-side session (Redis), rotate session id (fixation-safe)
 - INSERT user_sessions {login_time, session_date = shift_date(login_time)}
 - set cookies: HttpOnly, Secure, SameSite=Lax, 8h idle / 24h absolute
 - write AUDIT(LOGIN) via outbox
 - 302 /dashboard (page) | 200 {session summary} (API)

Active session → GET /logout

- find active session row (by session_id, server session – not "latest null logout")
- total_hours = (logout_time - login_time) / 3600
- AUDIT(LOGOUT); Redis session deleted (immediate revocation)

Forgot / reset password

POST /api/forgot-password {emp_id, email}

- always 202 "If the account exists, a reset link has been sent." [FR-AUTH-08]
- if match: token = urlsafe(32); store SHA-256(token), exp=+1h, used=false
- email {host}/reset-password?token=... (queued via outbox)

POST /api/reset-password {token, new_password}

- hash(token) lookup; used=false && not expired else 400
- new_password: length>=10 AND not in breached-password corpus else 400 [FR-AUTH-10]
- bcrypt/argon2 update; token.used=true; ALL other reset tokens for this user invalidated
- AUDIT(PASSWORD_RESET); all other active sessions for this user revoked

6.2 Break & Lunch approval workflow (corrected)

```
[None] --start(non-approval type)--> [Active] --end--> [Completed]
                                     |
                                     +--12h no end (scheduled job)--> [Orphaned]
                                     +--admin dispose--> [Completed, reason=admin]
```

Lunch (requires_approval=true):
Employee --POST /api/break-approvals {Lunch}-->

```

    unique(emp, break_type, shift_date, status='Pending') enforced by DB [CC-05]
    --> Pending --notify manager/HR-->
Manager/HR --POST ../approve (conditional UPDATE WHERE status='Pending')-->
    --> Approved --notify employee-->
Employee --POST /api/breaks/start {Lunch}-->
    REQUIRE an Approved approval for today's shift_date          *** corrected ***
    (v1.0 accepted Pending – Appendix A defect A-16... see A list; the approval step
    was therefore decorative)
    --> insert Active (auto-ends any other Active break first, same transaction)
Employee --POST /api/breaks/<id>/end--> Completed, duration computed

```

Approval routing (unchanged logic, now via CC-11 policy function):

```

target has manager AND actor==manager      → allow
target has manager AND actor is HR/Admin    → allow
target has manager AND actor is neither     → 403
target has no manager AND actor is HR/Admin → allow
target has no manager AND actor is not      → 403

```

6.3 Leave lifecycle (corrected)

```

Employee --POST /api/leaves {type, start, end, session}-->
    days = working_days(start, end, employee.weekly_off_pattern) - holidays_in_range [FR-
LEA-09]
    BEGIN TX:
        balance := SELECT ... FOR UPDATE (row lock)
        IF balance.remaining < days → ROLLBACK, 400 insufficient_balance
        overlap check → DB exclusion constraint (Pending/Approved, date range) 409 on violation
        INSERT leave_requests (Pending)
        UPDATE leave_balance SET reserved = reserved + days
    COMMIT
    --> notify manager (or active delegate, FR-LEA-08a)

Manager/HR/Admin --POST /approve (conditional UPDATE Pending->Approved)-->
    UPDATE leave_balance SET used_days = used_days + days, reserved = reserved -
days
    --> Approved + notify
    --POST /reject-->
    UPDATE leave_balance SET reserved = reserved - days
    --> Rejected + notify

Employee/Admin --POST /cancel (Pending, or Approved with future start)-->
    reverse reserved/used accordingly --> Cancelled

HR/Admin --POST /api/leave-grants {type, days, month, year, emp_ids[]}-->
    per employee: UPDATE ... SET total_days = total_days + days (upsert)
    INSERT grant row; notify; AUDIT(LEAVE_GRANT)

```

Holidays inside the leave range are excluded from days at every point they are counted (apply, approve, report) because all three call FR-LEA-09's single function.

6.4 Regularization (corrected)

```

Employee --POST /api/regularization {date, reason, requested_login_time?, requested_logout_t
->

```

```

future date → 400
unique(emp, date, status='Pending') → 409 on duplicate
--> Pending --notify manager/HR-->

```

```

Manager/HR --POST /approve (conditional UPDATE)-->
--> Approved
--> WRITE the corrected login/logout time into that day's session record *** corrected ***
--> trigger FR-JOB-01 recompute for that (employee, date)
--> notify
--POST /reject--> Rejected (attendance untouched) + notify

```

```

Employee/Admin --POST /cancel (Pending only)--> Cancelled
Admin --GET /export--> Excel

```

v1.0 approved regularizations without ever touching the attendance record, so an “approved” correction had no effect on hours, reports or payroll. That is closed here.

6.5 Onboarding — 5 steps (corrected)

Trigger: POST /api/offers/<id>/accept (see §6.7) – automatic, not a manual HR click

- candidate.status = Hired
- INSERT users {status='Pre-hire', allow_login=false, candidate_id}
- INSERT onboarding_workflow {current_step=1, step1_status='InProgress'}
- INSERT onboarding_checklist (5 required doc rows, Pending)
- generate a scoped pre-boarding link (signed token, 14-day expiry, no login required)

Step 1 – Offer Acceptance & Pre-boarding

New hire uses the pre-boarding token link (not a normal login) to:

- upload documents POST /api/preboarding/<token>/documents/<doc_type>
- validated via FR-D0C-02 pipeline; size gate per doc_type

Submit: >=1 required doc Uploaded → step1=Completed, step2=InProgress

- notify HR

Step 2 – Document Collection & Verification (HR)

HR reviews each doc: Approve / Reject (+ mandatory note)

- Rejected → notify new hire via the token link, doc reset to Pending
- All required docs Approved → gate open for step 3

Step 3 – System & Access Provisioning (IT, guarded)

Guard: step2 fully Approved

- creates IT task "Provision accounts & equipment"
- on task completion: users.allow_login = true, status = 'Active' (or 'Onboarding' if Day 1 has not arrived – configurable), first_login credentials issued
- step3 = Completed on task completion

Step 4 – Workstation & ID Card (Admin/IT)

Guard: step3 Completed

- tasks: desk allocation, ID card issue; marked by IT/Admin, not the new hire

Step 5 – Orientation & Buddy Assignment

Guard: step4 Completed

- buddy/manager assigned; orientation session scheduled
- COMPLETED BY the assigned buddy or manager confirming orientation happened
- (*** corrected: v1.0 had the employee "complete their own introduction" as the

```
terminal step, which is backwards – the org confirms the new hire was onboarded,
  not the reverse ***)
→ onboarding_workflow.completed = true; users.status = 'Active' (if not already)
→ notify HR, manager
```

6.6 Offboarding — corrected (parallel clearance, LWD-triggered revocation)

Trigger: POST /api/resignations {emp_id, notice_date, last_working_day, reason}
(by employee, or by HR for involuntary exits)
→ INSERT resignation record
→ HR/manager acknowledges → Stage 1 (Resignation & Acceptance) = Completed

Stage 2 (Knowledge Transfer & Manager Clearance) ┐ run in PARALLEL,
Stage 3 (Asset Return & IT Clearance) ┘ both gate Stage 4

Stage 2: manager confirms KT plan executed, sign-off recorded
 Stage 3: IT/Admin marks each issued asset Returned (FR-AST); gate requires
 outstanding assets(emp_id) == 0

Stage 4 (Full & Final Settlement) – guard: Stage 2 AND Stage 3 both Completed

- Finance computes F&F (pending payroll, LOP adjustment, leave encashment, deductions, asset damage charges if any)
- Finance approval required (maker-checker: preparer != approver)
- Stage 4 = Completed on Finance approval

```

Stage 5 (Exit Interview & Access Revocation)
→ exit interview scheduled/recorded (optional, does not block revocation)
→ SCHEDULED JOB (FR-JOB-04), nightly at 00:00 IST:
    for each resignation where last_working_day = today:
        users.allow_login = false
        all active sessions for that user revoked (Redis)
        user_permissions cleared
        users.status = 'Inactive'
        AUDIT(ACCESS_REVOKED, reason='LWD')
        notify_admins
(** corrected: revocation is time-triggered on LWD, not dependent on someone
remembering to click "complete offboarding" – v1.0's access revocation only
happened if and when stage 5 was manually finished **)

```

6.7 Recruitment (ATS) – corrected

```

HR/Admin: POST /api/jobs {title,...} → Open
Candidate: POST /api/candidates {name, email, job_id} → Applied
HR: PUT /api/candidates/<id>/status {Screened|Interviewed|Rejected|Withdrawn}
    (Hired is NOT a directly settable status – enforced by the state machine)
HR: POST /api/interviews {candidate_id, scheduled_at} → Scheduled
HR: PUT /api/interviews/<id>/feedback → Completed
HR: POST /api/offers {candidate_id, offered_salary, basic_pct, hra_pct, allowances_pct}
    → validate basic_pct + hra_pct + allowances_pct == 100          *** corrected ***
    → Pending; candidate.status = Offered

```

Candidate accepts (portal or HR-recorded):
 POST /api/offers/<id>/accept
 → 404 if offer missing; 409 if not Pending *** corrected ***

```
→ offer.status = Accepted, accepted_at = now
→ candidate.status = Hired
→ DOMAIN EVENT CandidateHired → triggers $6.5 onboarding creation automatically
*** corrected ***
```

Candidate declines / HR withdraws:

```
POST /api/offers/<id>/reject
→ 404 if missing; 409 if not Pending *** corrected ***
→ offer.status = Rejected; candidate.status = Rejected (or back to Interviewed
  if HR chooses to re-offer – explicit action, not implicit)
```

```
Pipeline view: GET /api/pipeline → per-job stage counts +
                converted {pre_hire, onboarding, active} via onboarding_workflow join
Stages: Applied → Screened → Interviewed → Offered → Hired
                \----- Rejected/Withdrawn (any stage before Hired) -----/
```

6.8 Payroll run — corrected

```

Finance: manage salary_structures (effective-dated, non-overlapping)
Finance: manage payroll_rates (effective-dated)
Finance: POST /api/salary-structures/preview → calc_payroll(...) – SAME function as the run
Finance: POST /api/payroll-runs {month, year}
    → unique(month, year, status != 'Cancelled') else 409
    → ENQUEUE background job (CC-09); response 202 {run_id, status:'Queued'}
    → worker: BEGIN TX
        employees := Active/Onboarding users with a salary structure effective
                    for the period *** corrected:
                    ALL such employees, not just role='Employee' ***
    for each employee:
        lop_days := attendance_days WHERE status IN ('Absent','Half-day')
                    for the period (from FR-JOB-01, not recomputed ad hoc)
        reimbursements := SUM(expense_claims WHERE status='Approved'
                                AND not yet paid AND emp_id=employee)
        item := calc_payroll(structure, rates, lop_days, reimbursements)
        INSERT payroll_items (status derived from run)
COMMIT → run.status = 'Draft'

```

```

Finance A: POST /api/payroll-runs/<id>/submit → Draft → Submitted
Finance B (≠ Finance A): POST /api/payroll-runs/<id>/approve → Submitted → Approved
                                *** new: maker-checker ***
System/Finance: POST /api/payroll-runs/<id>/finalize → Approved → Finalized (immutable)
                → payslip PDFs generated (background job), bank file, TDS report available
Correction after Finalized → new adjustment run referencing the original run_id,
                never an edit to Finalized items.

```

6.9 Expenses & tickets

```

Expenses:
  Employee: POST /api/expenses {cat_id, amount, description, receipt}
    emp_id := session.emp_id (ALWAYS – request body emp_id ignored) *** corrected ***
    → Pending
  Manager/HR: PUT .../status {Approved|Rejected}
    actor != claim.emp_id (no self-approval) AND (actor==manager OR HR/Admin)
    else 403
  Finance: PUT .../status {Paid} *** corrected:

```

only role=Finance, and only from Approved ***
 Every transition: conditional UPDATE + AUDIT + notify(category=Expenses)

Tickets:

Employee: POST /api/tickets {queue, sub_category, subject} → Open
 → notify_department(queue) + Super Admin
 Visibility (list AND detail, both server-enforced): owner OR assigned_to OR
 dept==category OR admin
 PUT .../status {Open→In Progress→Resolved→Closed}; Closed + new comment from
 reporter within 7 days → Reopened
 PUT .../assign (admin) → assigned_to; AUDIT

6.10 Holidays & opt-in

Unchanged from v1.0 in substance; approval/opt-in uniqueness is now a database constraint rather than an application-only check (CC-05).

6.11 Shift & daily attendance status

shift_date(dt, employee) = dt.date if dt.time >= employee.shift_start else dt.date - 1
 shift_start_dt(date, employee), shift_end_dt (rolled +1 day if shift_end < shift_start)

Nightly job (FR-JOB-01), per employee per shift date:

has_session := any user_sessions row for that shift_date
 has_approved_leave := leave_requests Approved covering that date
 is_holiday := holidays covering that date & location, National
 OR (Optional AND employee has an Approved opt-in)
 is_weekly_off := date's weekday ∈ employee.weekly_off_pattern
 *** corrected: was
 hard-coded Mon-Fri ***

status := 'Holiday' if is_holiday
 'On Leave' elif has_approved_leave
 'Weekly-off' elif is_weekly_off
 'Present' elif has_session AND shift_hours >= full_day_threshold
 'Half-day' elif has_session AND shift_hours >= half_day_threshold
 'Absent' else

UPSERT attendance_days {emp_id, date, status, shift_hours, source='job'}
 (a later Approved regularization for that date re-triggers this row only,
 via FR-REG-03)

6.12 Notification preferences

Unchanged in shape; “mark read” now sets is_read = true (FR-NOT-02) instead of deleting, and delivery goes through the outbox so a slow SMTP provider cannot block the originating request.

7. Data model and database constraints

7.1 Entity overview (PostgreSQL)

Core groupings, same domains as v1.0 with the additions marked **N**:

users 1--* user_sessions
 users 1--* shift_assignments (N, effective-dated)
 users 1--* breaks --> break_types
 users 1--* break_approvals --> break_types
 users 1--* leave_requests --> checked against leave_balance & holidays
 users 1--* leave_balance (adds `reserved` column, N)
 users 1--* leave_policy_assignments (N, effective-dated)
 users 1--* monthly_leave_grants
 users 1--* regularization_requests (adds requested_login/logout_time, N)
 users 1--* attendance_days (N – output of FR-JOB-01)
 users 1--* salary_structures (effective-dated, non-overlapping)
 users 1--* payroll_items --> payroll_runs
 payroll_runs --> payroll_approvals (N, maker-checker)
 users 1--* expense_claims --> expense_categories
 users 1--* tickets --> ticket_comments
 users 1--* documents
 users 1--* onboarding_workflow 1--* onboarding_checklist
 users 1--* resignations (N) 1--1 offboarding_workflow
 offboarding_workflow: stage2 & stage3 columns are independent (parallel)
 job_postings 1--* candidates 1--* interviews
 candidates 1--1 offer_letters (offered_salary + basic_pct/hra_pct/allowances_pct)
 candidates --0..1--> users (candidate_id), reachable only via offer acceptance
 holidays 1--* holiday_optins *--1 users
 audit_log (append-only; excluded from anonymisation for retention period)
 outbox_events (N – transactional outbox for email/notification/webhook delivery)
 idempotency_keys (N)

7.2 Key constraints (illustrative DDL, not exhaustive)

```

\CommentTok{--}{--} CC{-}01: identity
\KeywordTok{CREATE} \KeywordTok{SEQUENCE} \NormalTok{ emp\_code\_seq } \KeywordTok{START}
  ↳ \DecValTok{1} \NormalTok{;}
\KeywordTok{ALTER} \KeywordTok{TABLE} \NormalTok{ users } \KeywordTok{ALTER} \KeywordTok{COLUMN} \KeywordTok{id}
  ↳ \KeywordTok{SET} \KeywordTok{DATA} \KeywordTok{TYPE} \NormalTok{ bigint } \KeywordTok{GENERATED} \NormalTok{ }
  ↳ ALWAYS \KeywordTok{AS} \NormalTok{ IDENTITY;}

\CommentTok{--}{--} One Active break per employee (FR{-}ATT{-}02)
\KeywordTok{CREATE} \KeywordTok{UNIQUE} \KeywordTok{INDEX} \NormalTok{ uq\_one\_active\_break }
  ↳ \KeywordTok{ON} \NormalTok{ breaks (emp\_id) } \KeywordTok{WHERE} \NormalTok{ status } \OperatorTok{=}
  ↳ \StringTok{\textquotesingle}Active\textquotesingle \NormalTok{;}

\CommentTok{--}{--} One Pending Lunch approval per employee per shift date (FR{-}ATT{-}05)
\KeywordTok{CREATE} \KeywordTok{UNIQUE} \KeywordTok{INDEX} \NormalTok{ uq\_pending\_lunch\_approval }
  ↳ \KeywordTok{ON} \NormalTok{ break\_approvals (emp\_id, break\_type, break\_date) }
  ↳ \KeywordTok{WHERE} \NormalTok{ status } \OperatorTok{=}
  ↳ \StringTok{\textquotesingle}Pending\textquotesingle \NormalTok{;}

\CommentTok{--}{--} One Pending regularization per employee per date (FR{-}REG{-}02)
\KeywordTok{CREATE} \KeywordTok{UNIQUE} \KeywordTok{INDEX} \NormalTok{ uq\_pending\_regularization }
  ↳ \KeywordTok{ON} \NormalTok{ regularization\_requests (emp\_id, request\_date) }
  ↳ \KeywordTok{WHERE} \NormalTok{ status } \OperatorTok{=}
  ↳ \StringTok{\textquotesingle}Pending\textquotesingle \NormalTok{;}

\CommentTok{--}{--} No overlapping Pending/Approved leave for the same employee (FR{-}LEA{-}02)
\KeywordTok{CREATE} \NormalTok{ EXTENSION } \ControlFlowTok{IF} \KeywordTok{NOT} \KeywordTok{EXISTS} \NormalTok{ }
  ↳ btree\_gist;
\KeywordTok{ALTER} \KeywordTok{TABLE} \NormalTok{ leave\_requests } \KeywordTok{ADD}
  ↳ \KeywordTok{CONSTRAINT} \NormalTok{ no\_overlapping\_leave }
  ↳ \NormalTok{ EXCLUDE } \KeywordTok{USING} \NormalTok{ gist ( }
  ↳ \NormalTok{ emp\_id } \KeywordTok{WITH} \OperatorTok{=} \NormalTok{ }
  ↳ \NormalTok{ daterange(start\_date, end\_date, }
  ↳ \StringTok{\textquotesingle} \textquotesingle \NormalTok{) } \KeywordTok{WITH} \NormalTok{ }
  ↳ \NormalTok{ ) } \KeywordTok{WHERE} \NormalTok{ (status } \KeywordTok{IN} \NormalTok{ }
  ↳ \StringTok{\textquotesingle}Pending\textquotesingle \NormalTok{, } \StringTok{\textquotesingle}Approve\textquotesingle }
  ↳ d\textquotesingle \NormalTok{);}
  
```

```

\CommentTok{{-}} One leave\_balance row per employee/type/year (FR{-}LEA{-}06)}
\KeywordTok{ALTER} \KeywordTok{TABLE} \NormalTok{ leave\_balance } \KeywordTok{ADD}
  ↪ \KeywordTok{CONSTRAINT} \NormalTok{ uq\_balance } \KeywordTok{UNIQUE} \NormalTok{ ( emp\_id, leave\_type, }
  ↪ \DataTypeTok{year} \NormalTok{ )};}

\CommentTok{{-}} One payroll run per month/year unless cancelled (FR{-}PAY{-}05)}
\KeywordTok{CREATE} \KeywordTok{UNIQUE} \KeywordTok{INDEX} \NormalTok{ uq\_payroll\_period}
  \KeywordTok{ON} \NormalTok{ payroll\_runs (} \DataTypeTok{month} \NormalTok{ , } \DataTypeTok{year} \NormalTok{ )}
  ↪ \KeywordTok{WHERE} \NormalTok{ status } \OperatorTok{\textless} \textgreater{}
  ↪ \StringTok{\textquotesingle}Cancelled\textquotesingle} \NormalTok{ };}

\CommentTok{{-}} Non{-}overlapping salary structures per employee (FR{-}PAY{-}02)}
\KeywordTok{ALTER} \KeywordTok{TABLE} \NormalTok{ salary\_structures } \KeywordTok{ADD}
  ↪ \KeywordTok{CONSTRAINT} \NormalTok{ no\_overlapping\_structure}
  \NormalTok{ EXCLUDE } \KeywordTok{USING} \NormalTok{ gist (}
  \NormalTok{ emp\_id } \KeywordTok{WITH} \OperatorTok{=} \NormalTok{ ,}
  \NormalTok{ daterange(effective\_from, effective\_to, }
  ↪ \StringTok{\textquotesingle} \textgreater{} \textless{} \textquotesingle} \NormalTok{ ) } \KeywordTok{WITH} \NormalTok{ \&\&}
  \NormalTok{ );}

\CommentTok{{-}} One active onboarding checklist per employee (FR{-}ONB{-}03)}
\KeywordTok{CREATE} \KeywordTok{UNIQUE} \KeywordTok{INDEX} \NormalTok{ uq\_active\_onboarding}
  \KeywordTok{ON} \NormalTok{ onboarding\_workflow (emp\_id) } \KeywordTok{WHERE} \NormalTok{ completed}
  ↪ \OperatorTok{=} \KeywordTok{false} \NormalTok{ };}

\CommentTok{{-}} One active resignation/offboarding per employee (FR{-}OFF{-}01)}
\KeywordTok{CREATE} \KeywordTok{UNIQUE} \KeywordTok{INDEX} \NormalTok{ uq\_active\_resignation}
  \KeywordTok{ON} \NormalTok{ resignations (emp\_id) } \KeywordTok{WHERE} \NormalTok{ status}
  ↪ \OperatorTok{\textless} \textgreater{}
  ↪ \StringTok{\textquotesingle}Cancelled\textquotesingle} \NormalTok{ };}

\CommentTok{{-}} Offer split sums to 100 (FR{-}ATS{-}04)}
\KeywordTok{ALTER} \KeywordTok{TABLE} \NormalTok{ offer\_letters } \KeywordTok{ADD}
  ↪ \KeywordTok{CONSTRAINT} \NormalTok{ split\_sums\_100}
  \KeywordTok{CHECK} \NormalTok{ (basic\_pct } \OperatorTok{+} \NormalTok{ hra\_pct } \OperatorTok{+} \NormalTok{ }
  ↪ allowances\_pct } \OperatorTok{=} \DecValTok{100} \NormalTok{ );}

\CommentTok{{-}} One holiday opt{-}in per employee/holiday (FR{-}HOL{-}03)}
\KeywordTok{ALTER} \KeywordTok{TABLE} \NormalTok{ holiday\_options } \KeywordTok{ADD}
  ↪ \KeywordTok{CONSTRAINT} \NormalTok{ uq\_optin } \KeywordTok{UNIQUE} \NormalTok{ ( emp\_id, holiday\_id );}

\CommentTok{{-}} Money columns (CC{-}03)}
\CommentTok{{-}} e.g. ALTER TABLE payroll\_items ALTER COLUMN net TYPE numeric(14,2);}

\CommentTok{{-}} Optimistic locking (CC{-}08) – representative pattern applied to editable tables}
\KeywordTok{ALTER} \KeywordTok{TABLE} \NormalTok{ leave\_requests } \KeywordTok{ADD}
  ↪ \KeywordTok{COLUMN} \NormalTok{ version } \DataTypeTok{integer} \KeywordTok{NOT} \KeywordTok{NULL}
  ↪ \KeywordTok{DEFAULT} \DecValTok{1} \NormalTok{ };}

```

7.3 New tables (v2.0)

Table	Purpose
attendance_days	Output of the nightly finalisation job (FR-JOB-01); single source for reports and payroll LOP
shift_assignments	Effective-dated shift configuration per employee
leave_policy_assignments	Effective-dated leave policy per employee (accrual, carry-forward, weekly-off pattern)
approval_delegations	Manager → delegate, date range (FR-LEA-08a)
resignations	First-class resignation/notice record driving offboarding
payroll_approvals	Maker-checker trail for payroll runs
outbox_events	Transactional outbox for email/notification delivery (CC-09)
idempotency_keys	Stores response for repeated Idempotency-Key requests (CC-07)
mfa_credentials	TOTP secrets (encrypted at rest) for FR-AUTH-11

7.4 Retention and anonymisation

See §11.4. Each table above declares a retention class (transactional, statutory-7y, audit-indefinite-with-review) in the data dictionary maintained alongside the Alembic migrations.

8. API specification

- Base path `/api/v2/`. `/api/v1/*` (current v1.0 shape) is retained as a **deprecated alias** during migration (§14) and removed after the sunset date is announced.
- Auth: session cookie for the web app; **bearer token (short-lived JWT + refresh)** additionally supported for future mobile/integration clients.
- All mutating endpoints require X-CSRF-Token (cookie-based session) or the bearer token (token-based — CSRF not applicable).
- Idempotent-sensitive endpoints (§CC-07) accept Idempotency-Key.
- Error envelope per CC-13.
- OpenAPI 3.1 generated from Pydantic models (Pydantic + SpecTree/APIFlask), published at `/api/v2/openapi.json` and rendered at `/docs/`. Contract-tested with Schemathesis in CI.
- Endpoint groups mirror the FR modules in §5; the full generated catalogue (target **90+ routes**, exact count from CI, not hand-counted) lives in the OpenAPI document rather than being retyped here, to avoid the drift seen between v1.0's endpoint table and role matrix (Appendix A-27).

Representative corrected examples

POST `/api/v2/leaves`

Headers: Idempotency-Key: 3fa2... (CC-07)

Request:

```
{
  "leave_type": "Casual",
  "start_date": "2026-05-20",
  "end_date": "2026-05-21",
  "session": "Full",
  "reason": "Personal"
}
```

201:

```
{
  "leave_id": "01J...ULID",
  "days_requested": 2,
  "balance": {"total": 12, "used": 3, "reserved": 2, "remaining": 7},
  "status": "Pending"
}
```

400: {"error":{"code":"INSUFFICIENT_BALANCE","message":"...","details":{"remaining":1,"req"

409: {"error":{"code":"LEAVE_OVERLAP","message":"...","details":{"conflicting_leave_id":"..."}}

POST `/api/v2/payroll-runs`

202 Accepted:

```
{"run_id":"01J...","status":"Queued","poll":"/api/v2/payroll-runs/01J.../status"}
```

POST `/api/v2/offers`

Request: {"candidate_id":"...","offered_salary":900000,"basic_pct":50,"hra_pct":30,"allowan

400: {"error":{"code":"SPLIT_MUST_SUM_100","message":"basic_pct+hra_pct+allowances_pct must

9. UI / UX requirements

Retained from v1.0 in substance (Bootstrap 5, Inter font, slate palette, status chips, role-gated navbar) with these corrections:

ID	Requirement	Δ
UX-01	Live data (break status, notification bell, dashboard stats) updates via SSE , not client polling; UI still degrades to a manual refresh button if SSE is unavailable.	N
UX-02	Break button state is derived from a server call on page load and the SSE stream — never assumed from localStorage alone (the v1.0 self-heal pattern becomes the default source of truth, not a fallback).	C
UX-03	Onboarding pre-boarding pages work without a normal login , via the scoped token link (§6.5), on mobile-responsive layouts, since new hires may complete this from a phone before receiving a laptop.	N
UX-04	Payroll run, large exports and bulk import show a progress indicator backed by the job-status endpoint (§CC-09), not a blocking spinner.	N
UX-05	Forms surface field-level errors from the details object in the CC-13 error envelope.	C
UX-06	Accessibility: WCAG 2.2 AA, automated checks (axe-core) in CI on key pages (login, dashboard, leave apply, payroll run), not only manual review.	C

10. Non-functional requirements

Every NFR in v2.0 has a **measurable target** and a **verification method**; v1.0's NFR table mixed prose with numbers inconsistently.

Category	Requirement	Target	Verification
Performance	Page load (cold)	< 2 s at p95	Lighthouse CI
Performance	API latency	p95 < 300 ms, p99 < 800 ms, excluding async-job endpoints	k6 load test
Concurrency	Sustained load	150 req/s across the API for 15 minutes with error rate < 0.1%	k6, staging
Concurrency	Burst load	1,000 logins within a 5-minute shift-start window without lockouts caused by shared-NAT rate limiting (per-account, not per-IP-only)	k6 burst scenario
Scalability	Concurrent users	5,000 registered, 500 concurrently active sessions	Load test
Scalability	Horizontal scale	Web tier scales to N pods with zero code change (stateless)	Chaos test: kill a pod under load, verify no session loss (Redis)
Availability	Uptime	99.5% monthly (single-region, managed Postgres with automated failover)	Uptime monitoring
Availability	Recovery	RPO ≤ 15 min, RTO ≤ 2 h	Documented DR runbook, tested quarterly
Security	See §11	—	ASVS checklist, annual pen test
Usability	WCAG 2.2 AA on key flows	0 critical axe-core violations	CI

Category	Requirement	Target	Verification
Reliability	Scheduled jobs	Exactly-once execution across all pods (FR-JOB-05)	Chaos test with 3 scheduler-capable pods running
Reliability	Payroll run	Idempotent re-run for the same period produces identical results or a clear adjustment run, never duplicate items	Automated test
Maintainability	Migrations	Zero-downtime (expand/contract) for schema changes	Alembic + deploy checklist
Maintainability	Test suite	Unit, integration (testcontainers), contract (Schemathesis), load, and Playwright e2e, run separately in CI without the “shared DB file” workaround	CI pipeline
Observability	Every request traceable	request_id present in logs, traces and error responses end-to-end	OpenTelemetry trace sampling
Data integrity	Money	No floating-point money anywhere in code or schema	bandit/custom lint rule + code review checklist
Backup	Postgres	Automated daily full backup + WAL archiving, 30-day retention, quarterly restore drill	DR runbook
Portability	Deployment	docker compose up for local dev; Terraform for cloud; no code change between environments (12-factor)	CI build matrix

11. Security, privacy and compliance

11.1 Authentication & session

Argon2id password hashing (replaces bcrypt — Argon2 is the current OWASP recommendation); server-side sessions in Redis for instant revocation; MFA for privileged roles (FR-AUTH-11); account lockout (FR-AUTH-03); uniform error responses to prevent enumeration (FR-AUTH-02, FR-AUTH-08).

11.2 Authorization

Single policy function (CC-11) is the only place access decisions are made; page-level and API-level checks call the same function, closing the “API vs page 403/401 divergence” noted as a gap in v1.0 §11 SEC-02.

11.3 Application security controls

Control	v2.0 approach
CSRF	Enforced (double-submit token) on all mutating /api/* routes — fixes v1.0’s SEC-04 gap , where the token was issued but never checked
Security headers	Flask-Talisman: CSP (no inline scripts by default), X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy: strict-origin-when-cross-origin, HSTS in production
Rate limiting	Redis-backed, per-account and per-IP (not per-IP alone, which locks out shared-NAT offices)
Input validation	Pydantic models at every boundary; unknown fields rejected on writes

Control	v2.0 approach
File uploads	Content-sniffed MIME type, size caps per category, malware scan (ClamAV), random storage keys, presigned-URL access — fixes v1.0 SEC-10 and the extension-only check
SQL injection	SQLAlchemy parameterised queries exclusively; bandit in CI to catch raw string SQL
Authorization gaps closed	Document delete owner/admin only (was any-auth); expense Paid restricted to Finance (was any-auth); performance review submit restricted to the assigned reviewer (was any-auth); expense emp_id always from session (was body-overridable) — this closes every item listed in v1.0 §11 SEC-08
Secrets	Environment/secrets manager only; .secret_key file removed from the repo and from the deployment (CC-15)
Dependency and image scanning	pip-audit, Trivy, gitleaks in CI; blocked merge on high/critical findings
Audit	Every mutation audited via the outbox (FR-AUD-01); audit log is append-only and excluded from purge

11.4 Privacy and data retention

- The system processes personal data of Indian employees and is designed with the **Digital Personal Data Protection Act, 2023** in mind: purpose limitation, data minimisation on new fields, and a documented retention schedule per table (§7.3/7.4).
- **No hard purge of employees.** FR-USR-06 replaces v1.0's cascading delete with anonymisation, because payroll and audit records generally carry a statutory retention requirement (commonly cited as around 7 years for payroll/tax records in India) that a hard purge would violate. **The exact retention period per record type shall be confirmed with the organisation's legal/compliance function before go-live** — this SRS does not assert specific legal periods as fact.
- Data subject requests (access, correction) are handled through the profile (self-service) and an HR-mediated process for fields the employee cannot self-edit.
- PII fields (bank account, PAN, address, phone) are marked in the data dictionary and covered by pii_reveal permission (§2.2) with additional audit logging on read, not only on write.

11.5 Independent verification

An OWASP ASVS Level 2 self-assessment and an external penetration test are recommended before production go-live, and annually thereafter.

12. Reporting & analytics

Unchanged in intent from v1.0 (ops reports, admin analytics, department summaries), with two corrections: 1. "Working days" in every report uses the single function from FR-LEA-09, so leave, payroll and report figures agree with each other — v1.0 computed this three different ways. 2. Analytics endpoints read from materialized views (or DuckDB attached to the read replica) refreshed on a schedule, so heavy aggregation queries never compete with transactional traffic on the primary database.

13. Deployment, DevOps and CI/CD

Area	v2.0
Local dev Images	docker compose up — app, Postgres, Redis, MinIO, MailHog Multi-stage Dockerfile, non-root user, distroless or slim base, SBOM generated
Environments	dev → staging → production, identical images promoted, config via environment only
CI	GitHub Actions: lint (ruff), type-check (mypy), unit + integration tests (pytest + testcontainers, parallelised), contract tests (Schemathesis against the OpenAPI spec), security scans (bandit, pip-audit, Trivy, gitLeaks), Playwright e2e against a docker-compose stack, load test (k6) on a schedule/pre-release, not on every commit
CD	Blue/green or rolling deploy behind the load balancer; Alembic migrations run as an expand/contract pre-deploy step; automatic rollback on health-check failure
Scheduler	One dedicated, leader-elected scheduler process (Celery beat with a lock, or a managed cron), separate from web/worker pods (FR-JOB-05) — fixes v1.0's documented but unresolved "APScheduler duplicates under multi-worker" caveat
IaC	Terraform for managed Postgres, Redis, object storage, container platform, DNS, TLS
Secrets	Cloud secrets manager (or SOPS+age for smaller setups); rotated on a schedule
Observability	Structured JSON logs with request_id; traces via OpenTelemetry; dashboards in Grafana; alerts on error rate, latency, queue depth, replication lag

14. Migration plan from v1.0

A phased cutover, since the running system (users, leave balances, payroll history) must not be lost.

Phase	Content
0 — Freeze & inventory	Freeze schema changes on the DuckDB system; export a full data dictionary; write the target Postgres schema and Alembic baseline migration.
1 — Dual-write disabled, batch migrate	One-time ETL: DuckDB → Postgres (DuckDB's Postgres extension or a scripted export/import), with row-count and checksum reconciliation per table. Fix data that violates new constraints (e.g. overlapping leave, duplicate pending approvals) with a manual cleanup pass before enabling constraints.
2 — Service layer cutover	Introduce the repository/service layering (§3.1) against Postgres behind a feature flag; run the existing Playwright/pytest suite against the new stack; keep the DuckDB path available for fast rollback.
3 — Cross-cutting rules	Apply CC-01 through CC-16 module by module, starting with Attendance/Breaks and Leave (highest concurrency risk), then Payroll, then the rest.
4 — New capabilities	Ship FR-JOB-01 (daily attendance), the corrected onboarding/offboarding flows, payroll maker-checker, and the Finance role.
5 — Cutover	Final delta sync, maintenance window, DNS/traffic switch, DuckDB file kept read-only as an audit fallback for a defined period.
6 — Decommission	Remove the /api/v1/* alias after the announced sunset date; archive the legacy DuckDB file per the retention schedule (§11.4).

Each phase has its own rollback point; phases 1-3 can run in a staging environment with production-shaped data for as long as needed before phase 5 is scheduled.

15. Verification: traceability, acceptance criteria, concurrency tests

- **Traceability.** Every FR-* ID maps to: OpenAPI operation ID → service method → migration(s) → test ID(s) → template (where applicable). Generated by CI from code annotations, not maintained by hand, to prevent the drift seen in v1.0 §14 (e.g. FR-ATT-10 pointed at content that duplicated FR-ATT-09).
- **Acceptance criteria.** Each FR's acceptance criteria are Given/When/Then scenarios stored with the test suite; this document states the rule, the repository states the proof.
- **Concurrency-specific tests (new requirement, not present in v1.0):**
 - Two simultaneous POST /leaves/<id>/approve calls from different managers → exactly one succeeds, one gets 409.
 - Two simultaneous POST /breaks/start {Lunch} calls for the same employee → exactly one Active row exists afterward.
 - Two simultaneous payroll run submissions for the same (month, year) → exactly one Draft run created.
 - Scheduler leader-election test: 3 scheduler-capable processes running, exactly one executes each scheduled job per period.
 - Load test scenario matching the shift-start burst target in §10.

16. Risks and open questions

#	Risk / question	Owner to resolve
R-01	Exact PF/ESI/PT/TDS rates and slabs must be confirmed with Finance/CA before go-live; this SRS treats them as configuration, not as asserted current law.	Finance
R-02	Exact statutory retention period for payroll/audit records (§11.4) needs legal confirmation for the organisation's jurisdiction and sector.	Legal/Compliance
R-03	Whether a full FastAPI/React rewrite (§3.3) is preferred over evolving the Flask/jinja stack is a team-capability and roadmap decision, not a technical necessity.	Engineering leadership
R-04	Weekly-off pattern data does not exist in v1.0; it must be collected per employee/location before FR-JOB-01 and FR-LEA-09 can run correctly.	HR
R-05	SSO/OIDC (out of scope for v2.0) — confirm timeline, since it changes the auth module's design now rather than later.	Product/IT
R-06	Migration data-quality cleanup (Phase 1, §14) effort is unknown until the existing DuckDB data is profiled against the new constraints.	Engineering

17. Sign-off

Role	Name	Signature	Date
Product Owner			
Engineering Lead			
HR Stakeholder			
Finance Stakeholder			
Security Reviewer			

Role	Name	Signature	Date
QA Lead			

Appendix A — Defect register (v1.0 → v2.0)

Each item below was identified from a close reading of the v1.0 SRS text. They are written as **defects to verify against the running code**, since this review was document-based, not a code audit. Ticket the ones marked  as security-sensitive first.

ID	v1.0 area	Defect as described in v1.0	Corrected in v2.0
A-01	FR-AUTH-06 / SEC-01	Password minimum length is 6 characters.	FR-AUTH-10 — 10-char minimum, breached-password check
A-02 	SEC-04	CSRF token endpoint exists but v1.0's own security section lists enforcement as a gap ("recommend add").	FR-AUTH-12 — enforced on all mutating API routes
A-03 	FR-USR-06	Purge cascades and hard-deletes across 24+ tables, explicitly including audit and notifications.	FR-USR-06 — anonymisation replaces purge; audit/payroll retained
A-04	FR-REG-02	Create only takes {date, reason} — no corrected time captured.	FR-REG-02 — requires requested_login_time/requested_logout_time
A-05 	FR-REG-03	"Approve/Reject ... update + audit + notify" — no mention of writing to attendance/session data.	FR-REG-03 — approval writes the corrected time and triggers recompute
A-06	FR-LEA-04 / FR-PAY-04 / FR-RPT-01	Leave approval subtracts holidays "BETWEEN"; payroll doesn't mention LOP or holiday handling at all; reports compute working days via a separate Mon-Fri helper. Three different rules for "how many days".	FR-LEA-09 — one shared function
A-07	FR-PAY-02 / FR-PAY-05	SRS explicitly states runs use "a hard-coded variant — diverges from preview".	FR-PAY-02 — one calculation function used by both
A-08	FR-PAY-05	Payroll run joins "each role Employee" only — Admin, HR, IT, Team Leader are excluded from payroll by the stated logic.	FR-PAY-05 — all Active/Onboarding employees with an effective salary structure
A-09 	FR-PAY-06	Finalize is a single admin/HR action; no second approver.	FR-PAY-06 — maker-checker required
A-10 	FR-EXP-02	SRS explicitly notes "emp_id override from body allowed — treat as legacy".	FR-EXP-02 — emp_id always from session
A-11 	FR-EXP-03 / SEC-08	"Paid currently bypasses manager check (allow any logged)" — stated as a known gap.	FR-EXP-03 — Paid restricted to Finance role, from Approved only
A-12	FR-DOC-02	Upload validation is extension + size only.	FR-DOC-02 — MIME sniffing + malware scan added
A-13 	FR-DOC-03 / SEC-08	"Delete - currently any auth (gap: should be owner/admin)" — stated as a known gap.	FR-DOC-03 — owner or Admin only
A-14	FR-ONB-01	Pre-hire created with allow_login=0, yet Step 1 requires the same employee to log in and upload documents.	FR-ONB-01 — scoped pre-boarding token link, no login required

ID	v1.0 area	Defect as described in v1.0	Corrected in v2.0
A-15	FR-ONB-05	Step 5 “Complete intro” is a self-service action by the employee, described as the terminal step of their own onboarding.	FR-ONB-05 — buddy/manager confirms orientation, not the new hire
A-16	§6.6 (v1.0)	Offboarding steps are described strictly sequentially (1→2→3→4→5), which delays settlement until clearance and asset return finish one after another even when they could run together.	FR-OFF-01 — stages 2 and 3 run in parallel
A-17	FR-OFF-03	Access revocation only happens “step5 ⇒ completed=1 + users.status Inactive” — dependent on someone finishing stage 5 manually, no date-driven trigger.	FR-OFF-03/FR-JOB-04 — revoked automatically at 00:00 IST on last working day
A-18	FR-PERF-02 / SEC-08	“Submit PUT ... (auth any - no role check gap - record)” — stated as a known gap.	FR-PERF-02 — only the assigned reviewer may submit
A-19	FR-ATS-02/03	PUT /api/candidates/<id>/status {status in STAGES} allows setting Hired directly; convert is a separate manual step.	FR-ATS-03/04 — Hired reachable only via accepted offer
A-20	FR-ATS-03	Offer-accepted salary split is fixed at “basic50% hra20% allow20%” — sums to 90%, not 100%.	FR-ATS-04 — percentages validated to sum to 100
A-21	FR-ATS-04	“Reject ⇒ Rejected (idempotent even if missing)” — a reject/accept on a non-existent or already-decided offer returns success.	FR-ATS-04 — 404/409 on missing or already-decided offers
A-22	FR-RPT-01	Working-days helper is described as “Mon-Fri minus weekday holidays” — no weekly-off pattern per employee/location.	FR-LEA-09/FR-JOB-01 — per-employee weekly-off pattern
A-23	§3.4 / Roadmap 16.4.7	v1.0’s own architecture section flags “APScheduler in-process (avoid duplicate in multi-worker - TODO migrate to external cron)” as an unresolved caveat.	FR-JOB-05 — leader-elected single scheduler instance
A-24	FR-ANL-04	Attrition-risk formula weights (0.4, 1.5, 0.8, 3) are stated as fixed values in the requirement text.	FR-ANL-02 — weights are configuration, reviewed with HR
A-25	FR-RPT-01	Self-service restriction is described as “test-enforced” in v1.0’s own text — implying it depends on a test catching a regression rather than being structurally guaranteed.	FR-RPT-01 retained, explicitly re-stated as a server-side substitution rule, not just a filter
A-26	FR-NOT-01	“Mark read POST /api/notifications/read ... deletes own rows (misnomer - not mark but clear)” — stated as a naming/behaviour mismatch in v1.0 itself.	FR-NOT-02 — sets is_read = true, does not delete
A-27	§14 / §8.1	Traceability matrix is hand-maintained and partial (“excerpt”); separately, document metrics (34+ tables, 85+ endpoints, 258 tests) are stated as fixed counts on the cover page, which will drift from the code by definition.	§8 / §15 — generated from CI, not hand-counted or hand-typed

ID	v1.0 area	Defect as described in v1.0	Corrected in v2.0
A-28	§3.2 / §13	Environment constraint says “Python 3.10+” while pyproject target is py313, and CI is stated as “Not present” while a lint tool config already targets a specific Python version — an inconsistency between stated environment and stated tooling.	§3.2/§13 — Python 3.13, CI defined in §13

Appendix B — Role-permission matrix (target)

25 permission modules (21 from v1.0 plus payroll_approve, shift_admin, policy_admin, pii_reveal). “Self” = own records only (emp_id = session.emp_id, enforced server-side per CC-10/CC-11); Admin/Super Admin always see all; overrides in user_permissions can flip any cell per user.

Module group	Super Admin	Admin	HR	Finance	IT	Team Leader	Employee
users, import_users	✓	✓	candidates/onboarding only	–	–	–	–
jobs, candidates, offers (ATS)	✓	✓	✓	–	–	–	–
payroll, salary_structures, payroll_rates	✓	read-only	read-only	✓	–	–	own payslip
payroll_approve (N)		–	–	✓ (2nd approver ≠ preparer)	–	–	–
leaves, regularization, breaks, holidays	✓	✓	✓	–	–	leaves/reg for reports, own team approvals	self
expenses	✓	✓	approve only	Paid only	–	approve own team	self
onboarding, offboarding, audit, analytics, reports	✓	✓	✓	reports (payroll-related)	reports	reports, perf	reports (self)
tickets, assets	✓	✓	HR queue	–	✓	–	own
performance	✓	✓	–	–	–	own reports' goals	own goals
documents	✓	✓	✓	–	✓ (IT docs)	–	own
shift_admin (N)	✓	✓	–	–	–	–	–
policy_admin (N — leave/holiday policy config)	✓	✓	✓	–	–	–	–
pii_reveal (N — bank, PAN, address on read)	✓	✓ (audited)	✓ (audited)	✓ (audited, payroll-linked fields)	–	–	self

Appendix C — Glossary

Term	Meaning
ATS	Applicant Tracking System
CC-xx	Cross-cutting engineering rule (§4)
F&F	Full and Final settlement (offboarding)

Term	Meaning
FR-xx	Functional requirement identifier
IST	Indian Standard Time (Asia/Kolkata)
LOP	Loss of Pay (payroll deduction for unapproved absence)
LWD	Last Working Day
Maker-checker	The person who prepares/submits an action cannot be the person who approves it
Outbox pattern	Writing side-effect events (email, notification) in the same DB transaction as the business change, delivered afterward by a worker, so the two never disagree
RBAC	Role-Based Access Control
Shift date	The logical work day, keyed to shift start time, so a night shift belongs to one date instead of splitting across midnight
TDS	Tax Deducted at Source

End of document. Regenerate the traceability matrix and endpoint/test counts from CI on every release; do not hand-edit §8/§15 counts.